Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	05 / Modular Programming

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Fungsi, Argument, dan Parameter

```
contoh.py > ...
1   nama = input ("Masukkan nama: ")
2   print ("Hai", nama, "selamat datang!")

PROBLEMS    OUTPUT    TERMINAL    ...

PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\m
LENOVO\OneDrive\valine\Prak Alpro\minggu05\cont
Masukkan nama: valine
Hai valine selamat datang!
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\m
```

Gambar 1.1 : Contoh penggunaan fungsi **input** dan **print**.

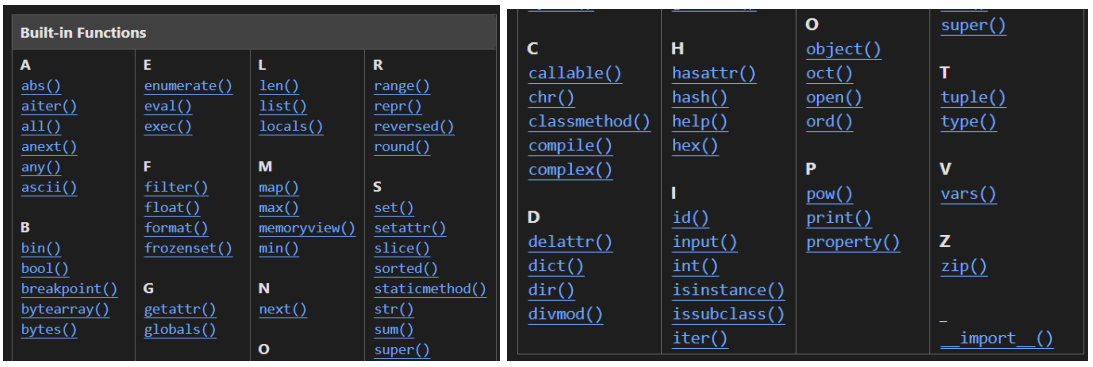
Pada contoh di atas, program meminta pengguna untuk memasukkan nama, kemudian program akan menyapa nama yang telah diinputkan oleh pengguna. Dalam program tersebut terdapat 2 fungsi yang digunakan, yaitu **input()** dan **print()**. Kedua fungsi tersebut adalah bawaan dari Python yang biasa disebut sebagai *built -in-function*.

- Fungsi **input()** = untuk membaca input yang diberikan oleh pengguna.
- Fungsi **print()** = untuk menampilkan tulisan di layar.

Apabila kita membuat program yang menggunakan banyak langkah, kita memerlukan untuk mengelompokkan beberapa kode program menjadi bagian-bagian (block) dari suatu program yang besar. **Modular** berarti keadaan dimana program terdiri dari beberapa bagian modular yang memiliki kegunaan khusus dan dapat digunakan ulang.

Berdasarkan asalnya, fungsi dibagi menjadi 2 jenis :

1) Fungsi bawaan (*built-in-function*).



Gambar 1.2 : Daftar fungsi bawaan Python3

(source : https://docs.python.org/3/library/functions.html)

2) Fungsi yang dibuat sendiri oleh programmer.

```
1  def tambah (a, b):
2      hasil = a + b
3      return hasil
```

Gambar 1.3 : Contoh fungsi **tambah()** buatan programmer.

Beberapa hal yang perlu diperhatikan :

- Keyword **def** = untuk mendefinisikan sebuah fungsi.
- Nama fungsi = **tambah()**
- Isi dari fungsi harus menjorok ke dalam 1 tab.
- Fungsi **tambah()** membutuhkan 2 argument (parameter **a** dan **b**).
- Fungsi akan menghasilkan hasil penjumlahan yang ditampung dalam sebuah variabel.
- Keyword **return** = untuk mengembalikan / mengeluarkan nilai dari suatu fungsi.

```
contoh.py > ...
1  def tambah (a, b):
2      hasil = a + b
3      return hasil
4
5  c = tambah (23, 25)
6  print(c)

PROBLEMS    OUTPUT    TERMINAL
PS C:\Users\LENOVO\OneDrive\va
LENOVO\OneDrive\valine\Prak Al
48
PS C:\Users\LENOVO\OneDrive\va
```

Gambar 1.3 : Contoh program

2. Return Value

Berdasarkan hasil yang dikeluarkan oleh fungsi, terbagi menjadi 2 jenis yaitu :

1) Fungsi yang tidak mengembalikan nilai (**void function**).
2) Fungsi yang mengembalikan nilai.

```
def print_twice(message):
    print(message)
    print(message)

print_twice("Halo Valine!")
```

Gambar 2.1 : Contoh fungsi yang tidak mengembalikan nilai.

- Fungsi print_twice() membutuhkan 1 parameter = message.
- Fungsi print_twice() akan menampilkan nilai dari variabel message sebanyak 2x.
- Fungsi print_twice() tidak menghasilkan suatu nilai yang dapat digunakan untuk proses berikutnya.
- Fungsi print_twice() akan menghasilkan nilai None jika dipanggil dengan cara berikut:

```
contoh.py > ...
def print_twice(message):
    print(message)
    print(message)

print (print_twice("Halo Valine!"))

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro
LENOVO\OneDrive\valine\Prak Alpro\minggu05\co
Halo Valine!
Halo Valine!
None
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro
```

Gambar 2.2 : Contoh program yang menghasilkan nilai None.

Fungsi **tambah()** yang ada di gambar 2.3 dapat dipanggil lagi untuk digunakan pada proses/langkah berikutnya.

```
contoh.py > ...
def tambah(a, b, c):
    hasil = a + b + c
    return hasil

nilai1 = 88
nilai2 = 79
nilai3 = 96

rata_rata = tambah(nilai1, nilai2, nilai3)/3
print(rata_rata)
```

Gambar 2.3 : Contoh penggunaan fungsi yang dapat digunakan lagi untuk proses selanjutnya.

3. Optional Argument dan Named Argument

Fungsi memiliki optional parameter yakni parameter yang bersifat opsional dan memiliki nilai bawaan (default) yang telah didefinisikan sebelumnya.

```
1  def hitung_belanja(belanja, diskon=0):
2      bayar = belanja - (belanja * diskon)/100
3      return bayar
```

Gambar 3.1 : Contoh fungsi yang memiliki optional parameter.

Pada fungsi hitung_belanja() ini terdapat 2 parameter = **belanja** dan **diskon**. Parameter **diskon** secara default memiliki nilai 0 (0%).

```
contoh.py > ...
1  def hitung_belanja(belanja, diskon=0):
2      bayar = belanja - (belanja * diskon)/100
3      return bayar
4
5  print(hitung_belanja(150000))
6  print(hitung_belanja(150000, 50))
7  print(hitung_belanja(150000, 25))

PROBLEMS   OUTPUT   TERMINAL   ...   Code  +

PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05>
LENOVO\OneDrive\valine\Prak Alpro\minggu05\contoh.py"
150000.0
75000.0
112500.0
```

Gambar 3.2 : Contoh cara untuk memanggil fungsi hitung_belanja()

Pada pemanggilan pertama hanya menggunakan 1 argument, sedangkan pada pemanggilan kedua dan ketiga menggunakan 2 argument.

```
contoh.py > ...
1  def cetak(a, b, c):
2      print("Nilai a: ",a)
3      print("Nilai b: ",b)
4      print("Nilai c: ",c)
5
6  cetak (20, 30, 40)
7  cetak (a = 20, b = 30, c = 40)
8  cetak (b = 30, c = 40, a = 20)
```

Gambar 3.3 : Beberapa cara memanggil fungsi cetak().

Pemanggilan fungsi dapat disertakan nama parameternya dan tidak perlu sesuai urutan yang diberikan. Pada line 8 dilakukan pemanggilan fungsi cetak() dengan menyebutkan nama argumennya (named argument), maka dari itu urutan argument tidak perlu sama dengan urutan parameter pada fungsi.

4. Anonymous Function (Lambda)

Anonymous function = fungsi tanpa nama (anonymous). Pada Python ini merupakan fitur tambahan, sedangkan pada bahasa pemrograman Haskell, Lisp, dan Erlang yang merupakan bahasa pemrograman fungsional. Pada Python, keyword Lambda digunakan untuk mendefinisikan anonymous function.

```
contoh.py > ...
1   tambah = lambda a, b: a + b
2   print (tambah(10,23))
```

Gambar 4.1 : Contoh penggunaan lambda.

Pada setiap anonymous function dalam Python pasti terdiri dari :

- Keyword : lambda
- Bound variable : argument pada lambda function
- Body : bagian utama lambda, berisi ekspresi / statement yang menghasilkan suatu nilai.

Sumber Materi : Modul 05 Modular Programming (05 - Modular Programming)

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Buatlah sebuah fungsi yang dapat menentukan apakah ketiga parameter memenuhi semua ketentuan :

- Ketiga parameter nilainya berbeda semua
- Ada kemungkinan jika diambil 2 parameter dan dijumlahkan hasilnya sama dengan parameter lainnya

Fungsi akan menghasilkan nilai True apabila semua ketentuan terpenuhi, jika tidak maka fungsi akan menghasilkan nilai False. Fungsi harus diberi nama cek_angka().

Source Code :

```
#fungsi cek angka

def cek_angka(a, b, c):

    if (a + b == c) or (a + c == b) or (b + c == a):

        return True

    else:

        return False


#testcase

print(cek_angka(2, 2, 4))

print(cek_angka(10, 4, 6))

print(cek_angka(8, 8, 8))

print(cek_angka(2, 3, 4))
```

Penjelasan :

1. Buat fungsi dengan nama cek_angka() seperti yang diminta oleh soal.
2. Masukkan tiga parameter (a, b, c) ke dalam nama fungsi hingga menjadi def cek_angka(a, b, c)
3. Untuk mengecek apakah ketiga parameter memiliki nilai yang berbeda seperti yang diminta, maka kita akan menggunakan percabangan.

4. Percabangan selanjutnya berarti membandingkan nilai angka antara a, b, dan c.
5. Dengan membuat 3 kemungkinan yang menentukan ketika 2 parameter dijumlahkan, maka hasil penjumlahan adalah parameter yang tersisa.
6. Jika parameter memenuhi 2 ketentuan tersebut, maka program akan mengembalikan nilai True, dan jika tidak, maka program akan mengembalikan nilai False.

Output :

soal01.py > ...

```python
#fungsi cek angka
def cek_angka(a, b, c):
    if (a + b == c) or (a + c == b) or (b + c == a):
        return True
    else:
        return False

#testcase
print(cek_angka(2, 2, 4))
print(cek_angka(10, 4, 6))
print(cek_angka(8, 8, 8))
print(cek_angka(2, 3, 4))
```

PROBLEMS OUTPUT TERMINAL ··· Code

```
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05> python
LENOVO\OneDrive\valine\Prak Alpro\minggu05\soal01.py"
True
True
False
False
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05>
```

SOAL 2

Source Code :

- **Versi Input dari User**

```
#pengguna perlu menginput bilangan yang akan dicek
a = int(input("Masukkan bilangan pertama = "))
b = int(input("Masukkan bilangan kedua = "))
c = int(input("Masukkan bilangan ketiga = "))

#fungsi untuk mengecek digit bilangan
def cek_digit_belakang(a, b, c):
    if (a % 10 == b % 10) or (a % 10 == c % 10) or (b % 10 == c % 10):
        return True
    else:
        return False

#menampilkan hasil dari inputan pengguna
print(cek_digit_belakang(a, b, c))
```

- **Versi Test Case**

```
#fungsi untuk mengecek digit bilangan
def cek_digit_belakang(a, b, c):
    if (a % 10 == b % 10) or (a % 10 == c % 10) or (b % 10 == c % 10):
        return True
    else:
        return False

#mengecek test case
print(cek_digit_belakang(30, 20, 18))
print(cek_digit_belakang(145, 25, 100))
```

```
print(cek_digit_belakang(71, 187, 18))
print(cek_digit_belakang(1024, 14, 94))
print(cek_digit_belakang(53, 8900, 658))
```

Penjelasan :

1. Soal menginginkan agar pengguna dapat menginput angka yang diinginkan, maka kita akan menggunakan fungsi input yang ditampung dalam variable a, b, dan c.
2. Buat fungsi dengan nama cek_digit_belakang() seperti yang diminta oleh soal.
3. Masukkan tiga parameter (a, b, c) ke dalam nama fungsi hingga menjadi def cek_digit_belakang(a, b, c).
4. Membuat percabangan yang membandingkan apakah digit paling belakang (paling kanan)nya sama atau tidak, dengan cara modulo (%) 10 agar dapat menemukan angka satuannya.
5. Ketika angka paling belakang sama (minimal 2 dari 3 parameter), maka program akan mengembalikan nilai True, dan jika tidak, maka akan mengembalikan nilai False.
6. Untuk mengecek test case yang diberikan oleh soal, saya telah membuat versi program untuk mengecek test case tersebut.

Output :

soal02.py > cek_digit_belakang

```python
#pengguna perlu menginput bilangan yang akan dicek
a = int(input("Masukkan bilangan pertama = "))
b = int(input("Masukkan bilangan kedua = "))
c = int(input("Masukkan bilangan ketiga = "))

#fungsi untuk mengecek digit bilangan
def cek_digit_belakang(a, b, c):
    if (a % 10 == b % 10) or (a % 10 == c % 10) or (b % 10 == c % 10):
        return True
    else:
        return False

#menampilkan hasil dari inputan pengguna
print(cek_digit_belakang(a, b, c))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05> python -u "c:\Users\LENOV
Masukkan bilangan pertama = 15
Masukkan bilangan kedua = 387
Masukkan bilangan ketiga = 295
True
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05>
```

soal02_testcase.py > ...

```python
#fungsi untuk mengecek digit bilangan
def cek_digit_belakang(a, b, c):
    if (a % 10 == b % 10) or (a % 10 == c % 10) or (b % 10 == c % 10):
        return True
    else:
        return False

#mengecek test case
print(cek_digit_belakang(30, 20, 18))
print(cek_digit_belakang(145, 25, 100))
print(cek_digit_belakang(71, 187, 18))
print(cek_digit_belakang(1024, 14, 94))
print(cek_digit_belakang(53, 8900, 658))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05> python -u "c:\Users\LENO\
True
True
False
True
False
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05>
```

SOAL 3

Source Code :

```python
#lambda function
celcius_to_fahrenheit = lambda C : (9/5) * C + 32
celcius_to_reamur = lambda C : 0.8 * C

print("Fahrenheit: ", celcius_to_fahrenheit(100))
print("Reamur: ", celcius_to_reamur(80))
print("Fahrenheit: ", celcius_to_fahrenheit(0))
```

Penjelasan :

1. Soal meminta untuk menggunakan fungsi lambda dan membuat variable bernama celcius_to_fahrenheit dan celcius_to_reamur dengan mengimplementasikan rumus konversi suhu yang telah diberikan.
2. Setelah membuat fungsi lambda, maka kita dapat menggunakan print untuk menampilkan hasil perhitungan.
3. Mengecek test case yang ada di soal.
4. Menambahkan keterangan Fahrenheit / Reamur dalam tanda petik ketika melakukan print supaya memperjelas output.

Output :

soal03.py > ...

```python
#lambda function
celcius_to_fahrenheit = lambda C : (9/5) * C + 32
celcius_to_reamur = lambda C : 0.8 * C

print("Fahrenheit: ", celcius_to_fahrenheit(100))
print("Reamur: ", celcius_to_reamur(80))
print("Fahrenheit: ", celcius_to_fahrenheit(0))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05> pyt
pro\minggu05\soal03.py"
Fahrenheit:  212.0
Reamur:  64.0
Fahrenheit:  32.0
PS C:\Users\LENOVO\OneDrive\valine\Prak Alpro\minggu05>
```